



The Maximum Subarray

[Problem](#) | [Submissions](#) | [Leaderboard](#) | [Discussions](#) | [Editorial](#) | [Topics](#)

We define subsequence as any subset of an array. We define a subarray as a contiguous subsequence in an array.

Given an array, find the maximum possible sum among:

1. all nonempty subarrays.
2. all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

arr = [-1, 2, 3, -4, 5, 10]

The maximum subarray sum is comprised of elements at indices [1, - 5]. Their sum is $2 + 3 + -4 + 5 + 10 = 16$. The maximum subsequence sum is comprised of elements at indices [1, 2, 4, 5] and their sum is $2 + 3 + 5 + 10 = 20$.

Function Description

Complete the maxSubarray function in the editor below.

maxSubarray has the following parameter(s):

- `int arr[n]`: an array of integers

Returns

- `int[2]`: the maximum subarray and subsequence sums

Input Format

The first line of input contains a single integer t , the number of test cases.

The first line of each test case contains a single integer n .

The second line contains n space-separated integers `arr[i]` where $0 \leq i < n$.

Constraints

- $1 \leq t \leq 10$
- $1 \leq n \leq 10^5$
- $-10^6 \leq arr[i] \leq 10^6$

The subarray and subsequences you consider should have at least one element.

Sample Input 0

```
2
4
1 2 3 4
6
2 -1 2 3 4 -5
```

Sample Output 0

```
10 10
10 11
```

Author
Difficulty
Max Score
Submitted By

sh5r3aknight
Medium
50
84126

NEED HELP?

- [View discussions](#)
- [View editorial](#)
- [View top submissions](#)

RESOURCES

- [Dynamic Programming Basics](#)

RATE THIS CHALLENGE



MORE DETAILS

- [Download problem statement](#)
- [Download sample test cases](#)
- [Suggest Edits](#)



```
2
3 public class Solution {
4
5     public static List<Integer> maxSubarray(List<Integer> arr) {
6
7         // Maximum subarray sum (Kadane's Algorithm)
8         int currentSum = arr.get(0);
9         int maxSubarray = arr.get(0);
10
11         // Maximum subsequence sum
12         int maxElement = arr.get(0);
13         int maxSubsequence = arr.get(0);
14
15         for (int i = 1; i < arr.size(); i++) {
16
17             int value = arr.get(i);
18
19             // Subarray
20             currentSum = Math.max(value, currentSum + value);
21             maxSubarray = Math.max(maxSubarray, currentSum);
22
23             // Subsequence
24             if (value > 0) {
25                 maxSubsequence += value;
26             }
27
28             maxElement = Math.max(maxElement, value);
29         }
30
31         // If all numbers are negative, choose the largest negative number
32         if (maxSubsequence <= 0) {
33             maxSubsequence = maxElement;
34         }
35
36         return Arrays.asList(maxSubarray, maxSubsequence);
37     }
38
39     public static void main(String[] args) {
40
41         Scanner sc = new Scanner(System.in);
42
43         int t = sc.nextInt();
44
45         while (t-- > 0) {
46
47             int n = sc.nextInt();
48
49             List<Integer> arr = new ArrayList<>();
50
51             for (int i = 0; i < n; i++) {
52                 arr.add(sc.nextInt());
53             }
54
55             List<Integer> result = maxSubarray(arr);
56
57             System.out.println(result.get(0) + " " + result.get(1));
58         }
59
60         sc.close();
61     }
62 }
63
```

Line: 62 Col: 2

```
58 }
59
60 sc.close();
61
62
63
```

Line: 62 Col: 2

 Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

Input (stdin)

[Download](#)

```
1 2
2 4
3 1 2 3 4
4 6
5 2 -1 2 3 4 -5
```

Your Output (stdout)

```
1 10 10
2 10 11
```

Expected Output

[Download](#)